

Backend Software Architecture and Data Model

University project documentation

Architecture overview

The backend is a layered application with four main responsibility groups:

- **Interface layer.** Request and response schemas, pagination envelopes, middleware, and validation contracts.
- **Application service layer.** Ingestion, demographic processing, codebook CRUD and parsing, codebook generation and application, analytical reads, export, linking, and deletion guards.
- **Domain and data-model layer.** SQLAlchemy-backed entities for corpora, documents, demographics, codebooks, themes, codes, jobs, runs, document-level coding, and assignments.
- **Infrastructure and cross-cutting layer.** Settings, logging, domain exceptions, temporary-upload cleanup, application settings, and the relational database.

Interface layer

The interface layer receives requests through middleware and validates data through Pydantic schemas. Request IDs and request logging are applied before validated operations are passed to application services.

Application service layer

Services contain the procedural backend behavior. They coordinate parsing, persistence, LLM and embedding calls, background jobs, analytical reads, exports, and deletion checks. They do not replace the persistent entities; they operate on them.

Domain and data-model layer

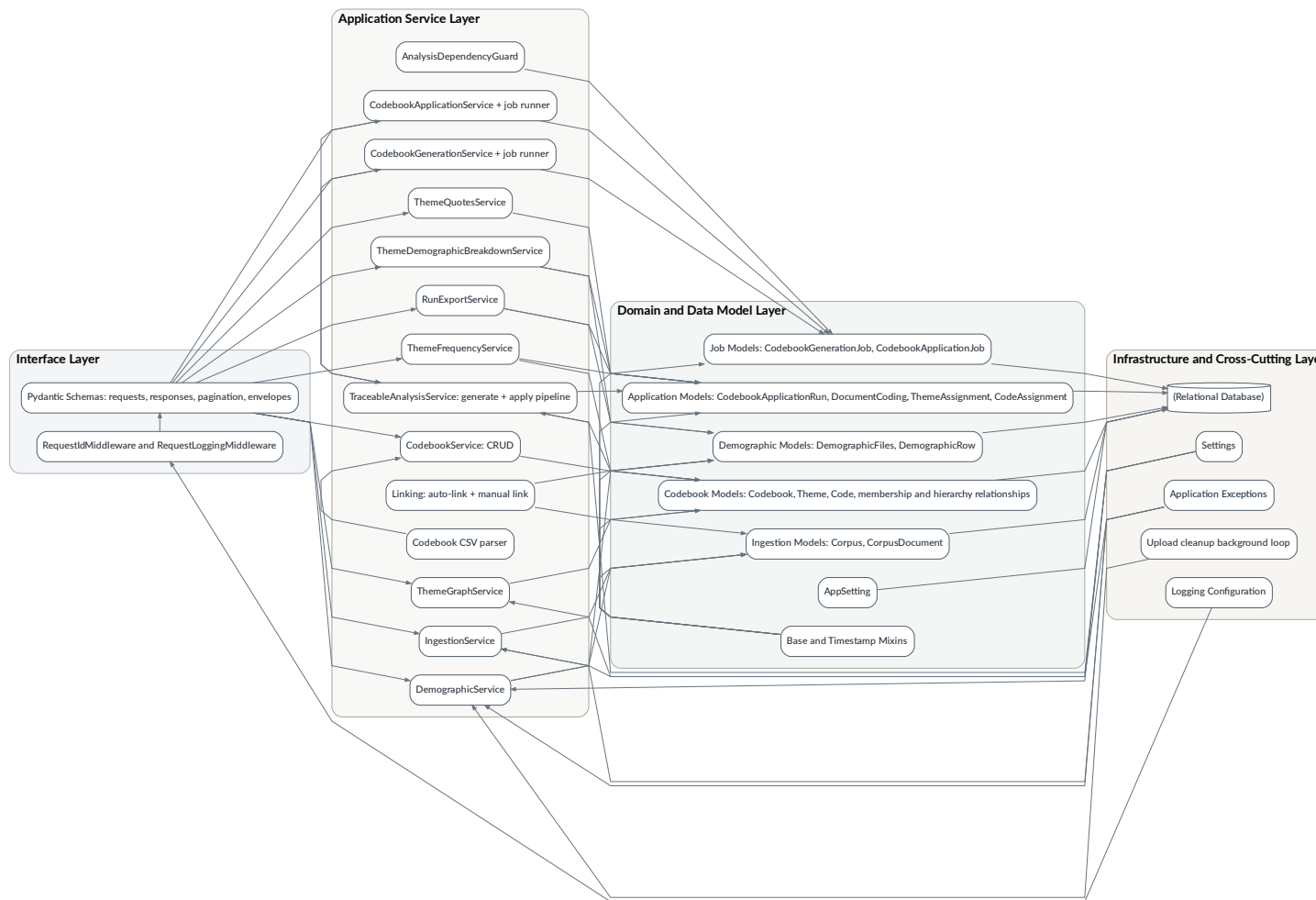
Persistent models hold source data, codebook structure, job state, application-run state, and evidence-bearing results. Base and timestamp mixins provide shared model fields.

Infrastructure and cross-cutting layer

Settings configure ingestion, demographic processing, and analysis. Logging supports middleware. Application exceptions represent known domain failures. The cleanup loop manages expired temporary uploads. Persistent models write to the relational database.

Dependency direction

Normal request flow moves from middleware and schemas to application services, then to domain models and the database. Infrastructure components provide configuration, logging, cleanup, and exception behavior across those layers.



Layered backend architecture. Arrows show control flow and persistence dependencies.

Direct dependencies

The diagram gives the main structure. The following table lists the complete direct dependency set represented by the current Mermaid definition.

Source	Direct target or targets
RequestIdMiddleware and RequestLoggingMiddleware Pydantic Schemas	Pydantic request, response, pagination, and envelope schemas. IngestionService; DemographicService; CodebookService; CodebookGenerationService; CodebookApplicationService; ThemeGraphService; ThemeFrequencyService; ThemeQuotesService; ThemeDemographicBreakdownService; RunExportService.
IngestionService DemographicService	Ingestion models: Corpus, CorpusDocument. Demographic models: DemographicFiles, DemographicRow; ingestion models: Corpus, CorpusDocument.
linking CodebookService	Ingestion models and demographic models. Codebook models: Codebook, Theme, Code, membership relationships, and hierarchy relationships.
codebook_parser CodebookGenerationService and job runner CodebookApplicationService and job runner TraceableAnalysisService	CodebookService. TraceableAnalysisService; CodebookGenerationJob. TraceableAnalysisService; CodebookApplicationJob. Codebook models and application-result models: CodebookApplicationRun, DocumentCoding, ThemeAssignment, CodeAssignment.
ThemeGraphService ThemeFrequencyService ThemeQuotesService ThemeDemographicBreakdownService RunExportService	Codebook models and hierarchy relationships. Application-result models and codebook models. Application-result models. Application-result models and demographic models. Application-result models and demographic models.
AnalysisDependencyGuard Ingestion models Demographic models Codebook models Job models Application-result models AppSetting Base and timestamp mixins Settings	Job models, primarily active CodebookApplicationJob records. Relational database. Relational database. Relational database. Relational database. Relational database. Relational database. Ingestion, demographic, codebook, job, and application-result models. IngestionService; DemographicService; TraceableAnalysisService.
Application exceptions Upload cleanup loop Logging configuration	IngestionService; DemographicService; ThemeGraphService. DemographicService and the temporary-upload filesystem area. Request middleware.

Service and component responsibilities

Service or component	Responsibility	Main data dependency
IngestionService	Creates, lists, and deletes corpora; ingests .txt, .docx, .pdf, and .jsonl documents; copies documents between corpora; can atomically create a new corpus from a selected document subset; uses AnalysisDependencyGuard before deletion.	Corpus, CorpusDocument
DemographicService	Imports demographic CSV data; confirms or cancels temporary uploads; lists and deletes demographic records; starts automatic linking to transcripts.	DemographicFiles, DemographicRow, CorpusDocument
linking	Automatically links transcripts to demographic rows by matching a document title against interviewee_id; also supports manual link and unlink.	CorpusDocument, DemographicRow
CodebookService	Atomically creates a codebook, its themes and codes, and all membership and hierarchy edges; reads and deletes codebooks.	Codebook, Theme, Code, relationship tables
codebook_parser	Parses a codebook CSV with node type, name, description, and parent name; produces typed NodeInput rows for THEME, SUBTHEME, and CODE; enforces a node count from 1 to 50 and checks hierarchy references.	In-memory; feeds CodebookService
TraceableAnalysisService	Extracts quote-grounded codes per document; consolidates near-duplicate codes using embeddings and LLM classification; synthesizes codes into subthemes and themes; reviews and refines the result against held-out documents; optionally applies the final codebook deductively to all documents. Every assignment is tied to a quote and character span.	Codebook and application-result models
CodebookGenerationService and job runner	Runs the traceable pipeline to generate a new codebook from transcripts; tracks asynchronous phase, progress, cancellation, timing, errors, and token usage.	CodebookGenerationJob
CodebookApplicationService and job runner	Applies an existing codebook to selected or all transcripts in its corpus; performs deductive coding only and delegates the document pass to TraceableAnalysisService.	CodebookApplicationJob, CodebookApplicationRun
ThemeGraphService	Builds and validates a directed acyclic tree of themes and codes within one codebook; enforces one parent and rejects cycles; uses anytree.	Theme, Code, hierarchy relationships
ThemeFrequencyService	Calculates theme occurrence counts and interview coverage percentages for one application run; rolls child occurrences up through parent themes.	ThemeAssignment, Theme, DocumentCoding
ThemeQuotesService	Returns a paginated, confidence-ranked list of quotes assigned to one theme within one application run.	ThemeAssignment, DocumentCoding, CorpusDocument
ThemeDemographicBreakdownService	Breaks theme presence down by one or more demographic dimensions and flags small samples.	ThemeAssignment, DocumentCoding, DemographicRow
RunExportService	Builds CSV exports for one application run in either theme-based or participant-based form.	Assignments, document coding, themes, documents, demographics
AnalysisDependencyGuard	Prevents deletion of documents, codebooks, or corpora that are referenced by queued or running application jobs unless force=True; forced deletion cancels affected jobs.	CodebookApplicationJob

Service or component	Responsibility	Main data dependency
app_settings	Reads and writes the active LLM provider. Resolution order is stored value, environment default, then FAU.	AppSetting
quote_matching	Locates an LLM-returned quote inside the transcript using exact, whitespace-normalized, then fuzzy matching; returns character offsets and a match status.	In-memory
remote_embeddings	Calls an OpenAI-compatible embeddings endpoint through the FAU gateway; prefilters similar code candidates before LLM consolidation.	External service; in-memory result
upload_cleanup	Runs at application startup and removes expired pending demographic-upload files from disk.	Filesystem

Main workflows

Corpus ingestion and demographic linking

1. A corpus is created or selected.
2. Source documents are parsed and stored as `CorpusDocument` records with their full text.
3. A demographic CSV is uploaded to temporary storage, parsed, and either confirmed or cancelled.
4. Confirmation creates one `DemographicFiles` record and one `DemographicRow` per imported row.
5. Auto-linking compares document titles with `interviewee_id`. A user can correct the result through manual link or unlink operations.

Codebook creation from CSV

1. The parser validates the expected columns, node types, hierarchy references, and the 1–50 node limit.
2. `THEME` and `SUBTHEME` rows become `Theme` records.
3. `CODE` rows become `Code` records.
4. Theme-to-theme and theme-to-code relationships are created in the same transaction as the codebook and nodes.

Generating a new codebook

1. A `CodebookGenerationJob` stores the corpus, requested name, optional research query and researcher topics, refinement limit, and whether to apply the codebook after generation.
2. The analysis service extracts quote-grounded codes from documents.
3. Embeddings prefilter likely duplicates; the LLM classifies and consolidates near-duplicate codes.
4. Codes are grouped into subthemes and themes.
5. The structure is reviewed and refined against held-out documents for up to the configured number of rounds.
6. The codebook and graph are persisted.
7. When `apply_after_generation` is enabled, the final codebook is applied to all documents and produces an application run.

Applying an existing codebook

1. A `CodebookApplicationJob` identifies the corpus, codebook, and selected document IDs. An empty or complete selection can represent all documents.
2. A `CodebookApplicationRun` is created as the durable run record.
3. Each document receives a `DocumentCoding` record.

4. Deductive coding produces theme-level and code-level assignments with confidence, evidence text, character offsets, and match status.
5. Job and run counters track completed and failed documents and accumulated token usage.

Analytical reads and export

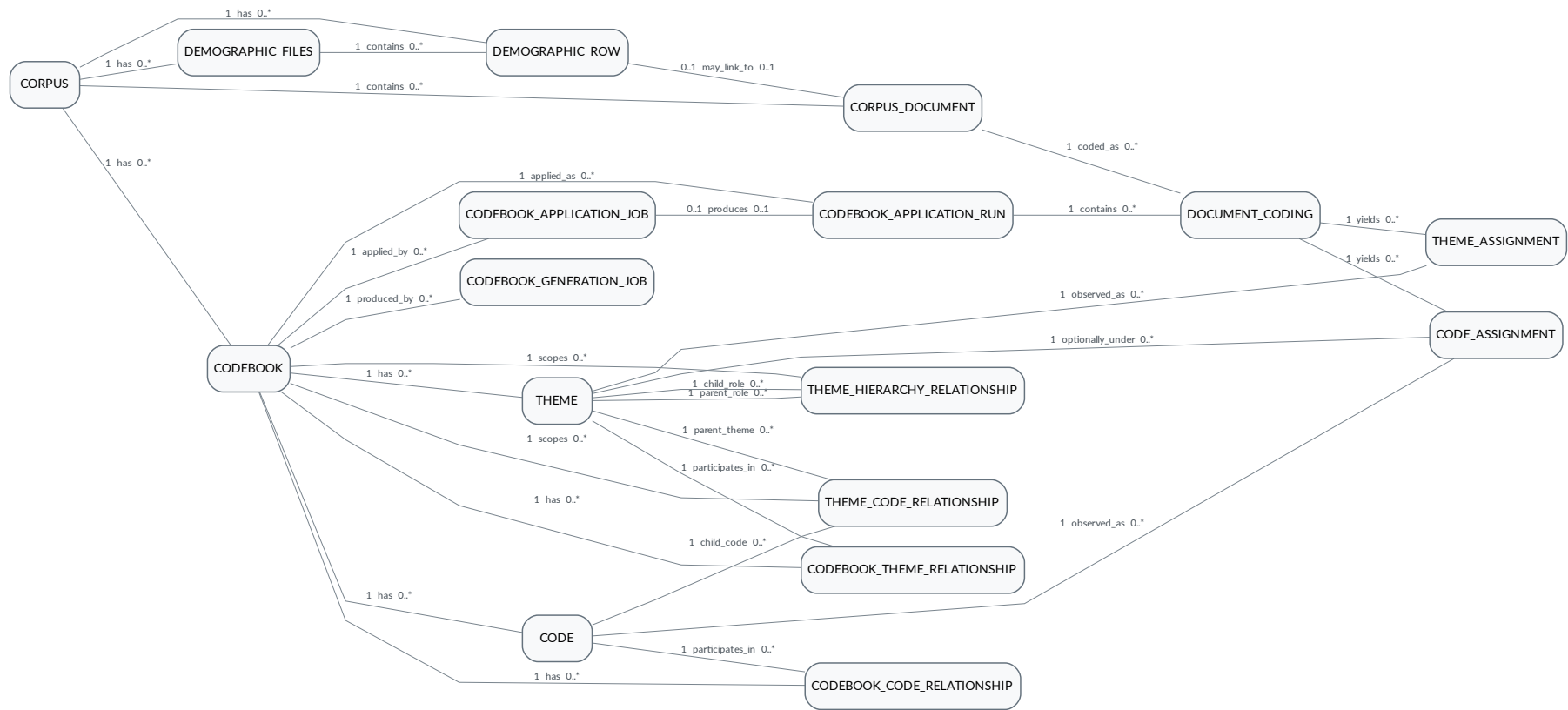
Theme graph reads reconstruct the validated codebook hierarchy. Frequency reads count occurrences and interview coverage. Quote reads return evidence sorted by confidence. Demographic breakdowns group theme presence by selected demographic fields and mark small groups. Exports produce theme-oriented or participant-oriented CSV files from one application run.

Safe deletion and cancellation

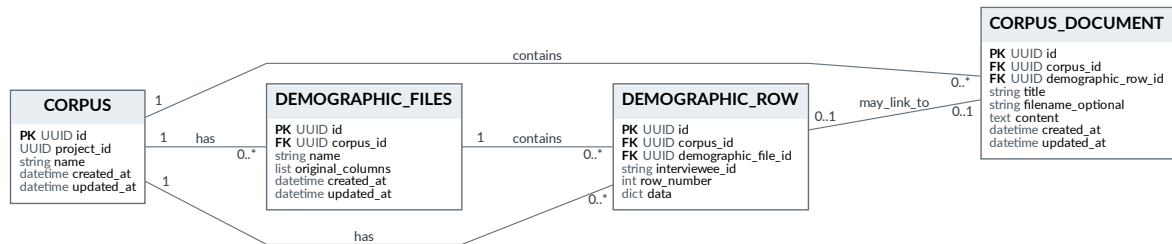
Before a corpus, document, or codebook is deleted, `AnalysisDependencyGuard` checks for queued or running application jobs. Deletion is blocked when active dependencies exist. With `force=True`, the affected jobs are cancelled and deletion can continue.

Persistent data model

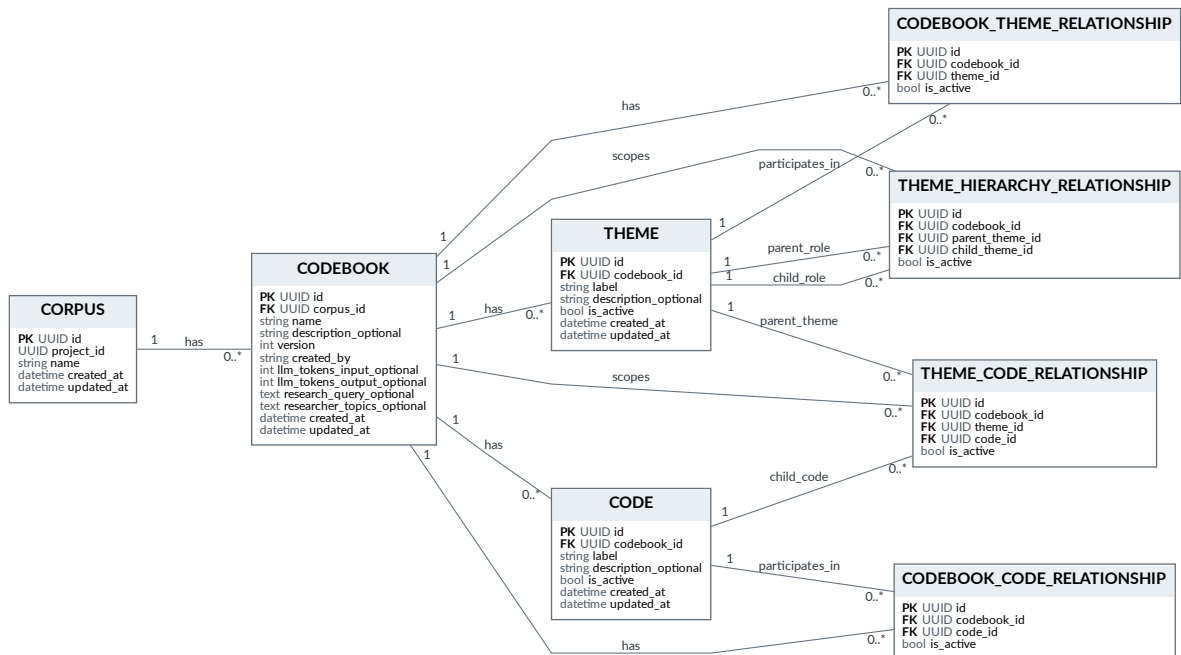
The model separates source material, demographic data, the codebook graph, asynchronous execution state, and coded results. The overview below shows the complete relationship structure. Focused views follow it so entity names and cardinalities remain readable.



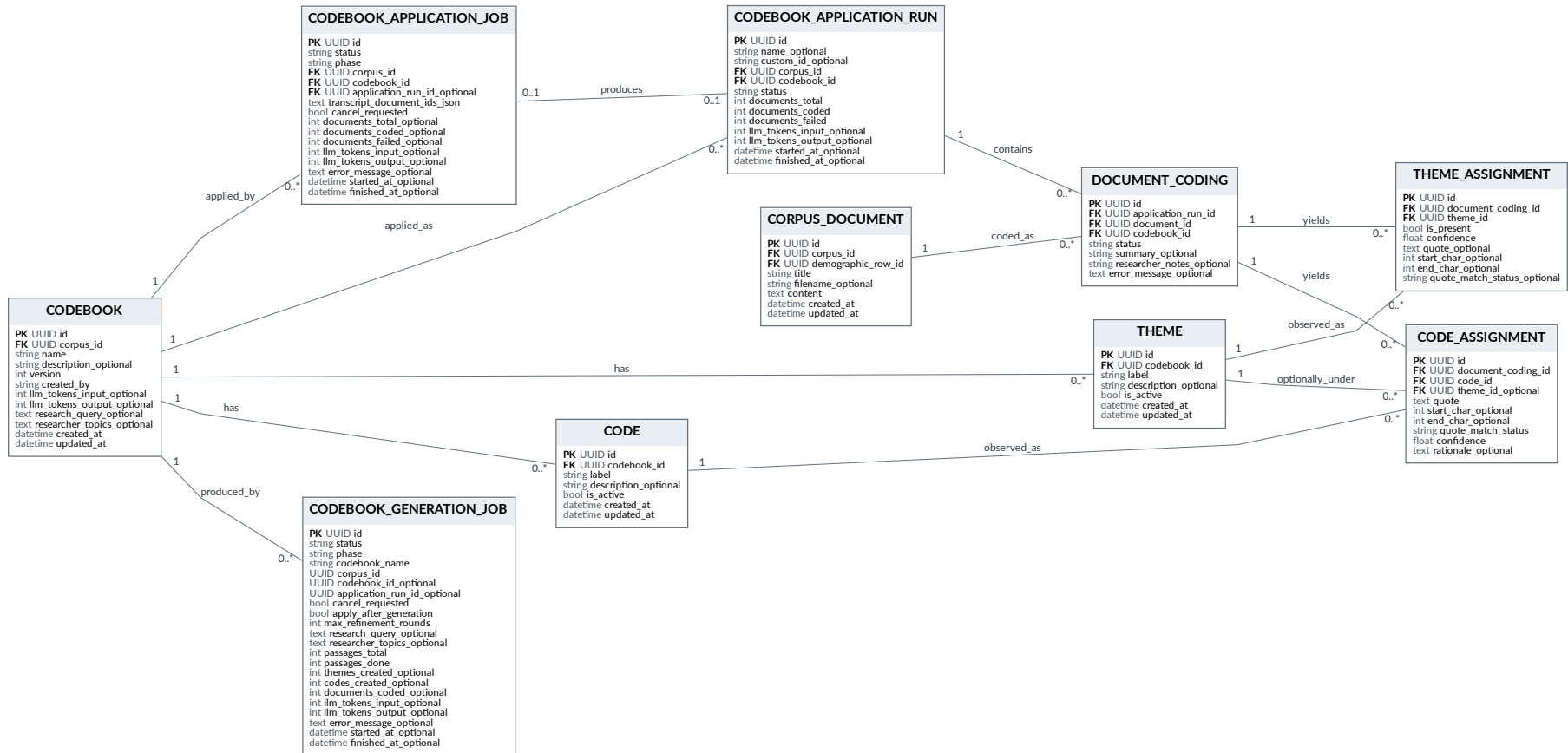
Complete conceptual entity-relationship model.



Corpus and demographic records.



Codebook membership and hierarchy graph.



Generation jobs, application jobs, runs, document results, and assignments.

Corpus and demographic domain

A Corpus contains source documents and owns demographic imports. A DemographicFiles record stores imported-file metadata and the original column list. Its DemographicRow records contain normalized row data. A demographic row can be linked to at most one corpus document, and a document can have at most one demographic-row link.

Codebook graph domain

A Codebook belongs to one corpus and owns themes and codes. Membership is represented both by direct foreign keys and explicit codebook membership tables. Theme hierarchy edges connect parent and child themes. Theme-code edges attach leaf-level codes to their parent theme or subtheme. ThemeGraphService treats this graph as a single-parent directed acyclic tree.

Execution and result domain

Generation jobs create new codebooks and can also produce an application run. Application jobs apply an existing codebook and produce one application run. A run contains one DocumentCoding record for each selected transcript. Each document result contains theme assignments and code assignments.

Settings domain

AppSetting is a key-value table. It is currently used to persist the active LLM provider selection across application restarts.

Relationship and integrity rules

- Every document, demographic file, demographic row, and codebook is scoped to a corpus.
- Every theme and code is scoped to a codebook.
- Hierarchy and membership edges are also scoped to a codebook, which prevents edges from joining nodes from different codebooks.
- Theme hierarchy is expected to be acyclic and single-parent.
- Codes are separate entities and are attached to themes through ThemeCodeRelationship.
- Application results reference the exact codebook used for the run and the exact document coded.
- Assignments belong to a document-coding record, not directly to a run, which keeps document-level status and errors separate.
- Optional foreign keys represent links that may not exist yet, such as a document-demographic link, a produced application run, or the optional theme context on a code assignment.

Entity catalogue

Entity	Purpose	Main fields
Corpus	Logical collection of source documents within a project.	id, project_id, name
CorpusDocument	One ingested document or transcript with metadata and complete text. It can be linked to one demographic row.	id, corpus_id, demographic_row_id, title, filename, content
DemographicFiles	Metadata for one imported demographic file, including its original columns.	id, corpus_id, name, original_columns
DemographicRow	One normalized imported demographic row.	id, corpus_id, demographic_file_id, interviewee_id, row_number, data

Entity	Purpose	Main fields
Codebook	Coding framework associated with a corpus. version is a simple per-corpus increment; simultaneous multi-version coexistence is not implemented.	id, corpus_id, name, description, version, created_by, research_query, researcher_topics
Theme	Theme or subtheme node in the codebook hierarchy. It does not represent a code.	id, codebook_id, label, description, is_active
Code	First-class leaf-level coding label, separate from Theme.	id, codebook_id, label, description, is_active
CodebookThemeRelationship	Explicit codebook-to-theme membership.	id, codebook_id, theme_id, is_active
CodebookCodeRelationship	Explicit codebook-to-code membership.	id, codebook_id, code_id, is_active
ThemeHierarchyRelationship	Directed parent-theme to child-theme edge.	id, codebook_id, parent_theme_id, child_theme_id, is_active
ThemeCodeRelationship	Directed theme-to-code edge.	id, codebook_id, theme_id, code_id, is_active
CodebookGenerationJob	Background state for generating a new codebook, optionally followed by application.	Identifiers, request parameters, status, phase, progress, token usage, error, timestamps
CodebookApplicationJob	Background state for applying an existing codebook to selected or all documents.	Identifiers, document selection, status, phase, progress, token usage, error, timestamps
CodebookApplicationRun	Durable record of one codebook application. It can be produced by either job type.	id, name, custom_id, corpus and codebook IDs, status, document counts, token usage, timing
DocumentCoding	Result and status boundary for one document within one application run.	id, run, document and codebook IDs, status, summary, researcher_notes, error_message
ThemeAssignment	Theme-level presence decision for one coded document with supporting evidence.	document coding and theme IDs, is_present, confidence, quote and offsets, match status
CodeAssignment	Span-level code assignment with evidence and optional rationale.	document coding, code and optional theme IDs, quote and offsets, match status, confidence, rationale
AppSetting	Generic single-row-per-key settings store.	key, value

Themes and codes are different node types

A CSV upload or LLM-generated codebook stores each node according to its declared type. THEME and SUBTHEME rows become Theme records connected through ThemeHierarchyRelationship. CODE rows become Code records connected to a parent theme through ThemeCodeRelationship. Themes support hierarchy, roll-up, and theme-level presence. Codes represent leaf-level labels and span-level assignments.

Traceability

Every ThemeAssignment and CodeAssignment stores the evidence used for the result:

- the supporting quote;
- start_char and end_char offsets into the source transcript;
- a confidence value;
- quote_match_status;
- an optional rationale for code assignments;
- is_present for theme-level presence decisions.

The quote matcher tries to locate returned evidence in this order:

1. exact text match;
2. whitespace-normalized match;
3. fuzzy match;
4. not_found when no reliable location can be produced.

The stored status is one of exact, normalized, fuzzy, or not_found. A reported result can therefore be followed from the application run to a document-coding record, then to an assignment, and finally to a specific text span in the original transcript. The same evidence model is used for generated codebooks and for existing codebooks applied deductively.

Jobs, runs, and lifecycle state

Generation jobs

CodebookGenerationJob stores the requested codebook name, corpus, optional research query and researcher topics, maximum refinement rounds, and whether the codebook should be applied after generation. Runtime fields record status, phase, cancellation request, passage progress, themes and codes created, documents coded, token usage, error text, and start and finish timestamps.

Application jobs

CodebookApplicationJob stores the corpus, codebook, selected transcript IDs, cancellation request, produced run, document totals, coded and failed counts, token usage, error text, and timestamps. Active application jobs are the dependencies checked before destructive operations.

Application runs and document status

CodebookApplicationRun is the durable record of one application. It stores optional human-readable identifiers, status, document totals, failures, token usage, and timing. DocumentCoding provides a separate status and error boundary for each transcript, so a partial run can keep successful results while recording failures on individual documents.

Cancellation and errors

Cancellation is requested through cancel_requested. The model does not imply immediate interruption at every instruction boundary; the job runner must check the flag at supported points. Errors are stored on jobs and document results. Separate coded and failed counters preserve partial-run outcomes.

Validation and safety controls

- **Interface validation.** Pydantic schemas define request, response, pagination, and envelope contracts.

- **Codebook upload checks.** The parser validates node types, hierarchy references, and a total node count from 1 to 50.
- **Atomic codebook creation.** Codebook nodes and graph relationships are created together.
- **Graph checks.** Theme reconstruction rejects cycles and multiple parents.
- **Correctable source linking.** Automatic title-to-interviewee matching can be corrected manually.
- **Evidence-location status.** Quote matching records whether the location was exact, normalized, fuzzy, or not found.
- **Deletion protection.** Active application jobs block dependent corpus, document, or codebook deletion unless forced cancellation is requested.
- **Operational records.** Jobs and runs keep progress, timing, failure, cancellation, and token-usage data.

Infrastructure and cross-cutting behavior

Configuration and provider selection

Application and environment settings influence service behavior. The active LLM provider resolves in this order: persisted setting, environment default, FAU fallback. The embeddings client uses an OpenAI-compatible interface through the FAU gateway.

Logging and request correlation

RequestIdMiddleware and RequestLoggingMiddleware associate service activity with inbound requests. Logging configuration is consumed by the middleware layer.

Application exceptions

Domain-specific exceptions are used by ingestion, demographic, and theme-graph operations. This allows request handlers to map known domain failures without embedding persistence rules in the interface layer.

Temporary-upload cleanup

A background loop started with the application deletes expired pending demographic-upload files. Confirmed demographic data is stored separately in the relational database.

Current limitations and boundaries

- `Codebook.version` is a simple per-corpus increment. The model does not implement simultaneous, independently managed codebook versions.
- The ER model is conceptual and derived from SQLAlchemy entities. Physical indexes, engine-specific types, database triggers, and migration details are not included in the source description.
- Deployment topology, authentication, authorization, network boundaries, and frontend structure are outside the documented model.
- `cancel_requested` is cooperative state. Actual cancellation timing depends on where job runners check it.
- Fuzzy quote matching preserves traceability status but does not guarantee that the selected span is semantically correct. Consumers should use the status and confidence fields when reviewing results.
- Automatic demographic linking is based on document-title and interviewee-ID matching and may require manual correction.

Attribute reference

Entity	Attributes
Corpus	id: UUID primary key; project_id: UUID; name: string; created_at: datetime; updated_at: datetime.
CorpusDocument	id: UUID primary key; corpus_id: UUID foreign key; demographic_row_id: optional UUID foreign key; title: string; filename: optional string; content: text; created_at: datetime; updated_at: datetime.
DemographicFiles	id: UUID primary key; corpus_id: UUID foreign key; name: string; original_columns: list; created_at: datetime; updated_at: datetime.
DemographicRow	id: UUID primary key; corpus_id: UUID foreign key; demographic_file_id: UUID foreign key; interviewee_id: string; row_number: integer; data: dictionary.
Codebook	id: UUID primary key; corpus_id: UUID foreign key; name: string; description: optional string; version: integer; created_by: string; llm_tokens_input: optional integer; llm_tokens_output: optional integer; research_query: optional text; researcher_topics: optional text; created_at: datetime; updated_at: datetime.
Theme	id: UUID primary key; codebook_id: UUID foreign key; label: string; description: optional string; is_active: boolean; created_at: datetime; updated_at: datetime.
Code	id: UUID primary key; codebook_id: UUID foreign key; label: string; description: optional string; is_active: boolean; created_at: datetime; updated_at: datetime.
CodebookThemeRelationship	id: UUID primary key; codebook_id: UUID foreign key; theme_id: UUID foreign key; is_active: boolean.
CodebookCodeRelationship	id: UUID primary key; codebook_id: UUID foreign key; code_id: UUID foreign key; is_active: boolean.
ThemeHierarchyRelationship	id: UUID primary key; codebook_id: UUID foreign key; parent_theme_id: UUID foreign key; child_theme_id: UUID foreign key; is_active: boolean.
ThemeCodeRelationship	id: UUID primary key; codebook_id: UUID foreign key; theme_id: UUID foreign key; code_id: UUID foreign key; is_active: boolean.
CodebookGenerationJob	id: UUID primary key; status: string; phase: string; codebook_name: string; corpus_id: UUID; codebook_id: optional UUID; application_run_id: optional UUID; cancel_requested: boolean; apply_after_generation: boolean; max_refinement_rounds: integer; research_query: optional text; researcher_topics: optional text; passages_total: integer; passages_done: integer; themes_created: optional integer; codes_created: optional integer; documents_coded: optional integer; llm_tokens_input: optional integer; llm_tokens_output: optional integer; error_message: optional text; started_at: optional datetime; finished_at: optional datetime.
CodebookApplicationJob	id: UUID primary key; status: string; phase: string; corpus_id: UUID foreign key; codebook_id: UUID foreign key; application_run_id: optional UUID foreign key; transcript_document_ids_json: text; cancel_requested: boolean; documents_total: optional integer; documents_coded: optional integer; documents_failed: optional integer; llm_tokens_input: optional integer; llm_tokens_output: optional integer; error_message: optional text; started_at: optional datetime; finished_at: optional datetime.
CodebookApplicationRun	id: UUID primary key; name: optional string; custom_id: optional string; corpus_id: UUID foreign key; codebook_id: UUID foreign key; status: string; documents_total: integer; documents_coded: integer; documents_failed: integer; llm_tokens_input: optional integer; llm_tokens_output: optional integer; started_at: optional datetime; finished_at: optional datetime.
DocumentCoding	id: UUID primary key; application_run_id: UUID foreign key; document_id: UUID foreign key; codebook_id: UUID foreign key; status: string; summary: optional string; researcher_notes: optional string; error_message: optional text.
ThemeAssignment	id: UUID primary key; document_coding_id: UUID foreign key; theme_id: UUID foreign key; is_present: boolean; confidence: float; quote: optional text; start_char: optional integer; end_char: optional integer; quote_match_status: optional string.
CodeAssignment	id: UUID primary key; document_coding_id: UUID foreign key; code_id: UUID foreign key; theme_id: optional UUID foreign key; quote: text; start_char: optional integer; end_char: optional integer; quote_match_status: string; confidence: float; rationale: optional text.
AppSetting	key: primary-key string; value: stored setting value.

Terms used in the model

Corpus

A collection of source documents and related demographic and codebook data.

Codebook

The complete coding framework for a corpus, including themes, subthemes, codes, and their graph edges.

Theme

A hierarchical grouping node used for higher-level interpretation and presence reporting.

Code

A leaf-level label used for span-level coding.

Generation job

Asynchronous work that creates a new codebook from source transcripts.

Application job

Asynchronous work that applies an existing codebook to selected transcripts.

Application run

The durable result container for one application of one codebook to a set of documents.

Document coding

The status and result boundary for one document in one run.

Assignment

A theme or code result tied to evidence in one document.

Traceable

Linked to a stored quote, character offsets, and a recorded quote-match status.